Análisis de código con IA en DevSecOps

Francisco Noceda

Abstract–
La adopción de metodologías ágiles, junto con las prácticas de integración y entrega continua, ha impulsado la automatización de los procesos de desarrollo de software. En este contexto, DevSecOps incorpora la seguridad a lo largo de todo el ciclo de vida de las aplicaciones con el objetivo de detectar vulnerabilidades de forma temprana y reducir riesgos sin afectar la velocidad de entrega. Sin embargo, actividades como el análisis de código, la creación de pruebas y la generación de documentación continúan requiriendo una importante intervención manual, lo que puede impactar en la eficiencia y calidad del desarrollo.

El análisis realizado demuestra el potencial de las herramientas de Inteligencia Artificial para complementar las prácticas DevSecOps tradicionales. Su utilización facilita la detección temprana de problemas de calidad y seguridad, agiliza actividades de soporte al desarrollo y contribuye a mejorar la eficiencia general del proceso de construcción de software.

Index Terms-
DevSecOps, Inteligencia Artificial, CI/CD, Seguridad del Software, Automatización, Análisis de Código, Testing Automatizado

I. Nomenclatura

LLM: Large Language Model
OWASP: Open Worldwide Application Security Project
CI/CD: Continuous Integration / Continuous Delivery (o Continuous Deployment)
IA: Inteligencia Artificial

II. Introducción

La creciente adopción de metodologías ágiles y prácticas de integración y entrega continua ha impulsado a las organizaciones a automatizar cada vez más sus procesos de desarrollo de software. En este contexto, DevSecOps surge como una evolución de DevOps que incorpora controles de seguridad a lo largo de todo el ciclo de vida de las aplicaciones, buscando reducir riesgos sin afectar la velocidad de entrega.

A pesar de los avances en automatización, muchas tareas relacionadas con el análisis de código, la creación de pruebas y la generación de documentación continúan requiriendo una intervención significativa por parte de los desarrolladores. Esto puede generar demoras y aumentar la probabilidad de que ciertos errores o vulnerabilidades pasen desapercibidos durante las primeras etapas del desarrollo.

Ante este escenario, el avance reciente de las herramientas basadas en inteligencia artificial abre nuevas posibilidades para complementar los procesos tradicionales de desarrollo y seguridad, permitiendo automatizar tareas complejas y asistir a los equipos de software en distintas fases del ciclo de desarrollo.

III. Caso de estudio

Se plantea como caso de estudio una empresa ficticia de desarrollo de software que construye soluciones web para el sector financiero, incluyendo plataformas de gestión de pagos y administración de cuentas utilizadas por bancos y empresas fintech. Debido a la naturaleza sensible de los datos que procesa, la organización debe cumplir con altos estándares de seguridad, calidad del código y confiabilidad del software.

El equipo de desarrollo está compuesto por aproximadamente 20 desarrolladores, quienes trabajan sobre una aplicación web basada en microservicios. El desarrollo se gestiona mediante un repositorio Git, donde los cambios son integrados a través de commits y pull requests.

La organización cuenta además con dos áreas especializadas que participan en el ciclo de vida del software:
-Equipo de Seguridad, encargado de analizar vulnerabilidades y validar que el software cumpla con los estándares de seguridad requeridos.
-Equipo de Infraestructura, responsable de la gestión de entornos, despliegues y mantenimiento del pipeline de integración y despliegue continuo.

Para automatizar el proceso de integración y entrega del software, la empresa utiliza un pipeline DevSecOps que se ejecuta automáticamente cada vez que se realizan cambios en el repositorio.
El pipeline actual contempla distintas etapas automatizadas que abarcan el ciclo de integración y entrega del software.

En primer lugar, se realiza el **build** del proyecto, donde se compila la aplicación y se verifican posibles errores de construcción.

Posteriormente, se ejecutan procesos de **análisis de seguridad** mediante herramientas de escaneo estático, orientados a identificar vulnerabilidades y malas prácticas en el código.

A continuación, se llevan a cabo las **pruebas automatizadas**, que permiten validar el correcto funcionamiento de los

distintos componentes del sistema.

Finalmente, el pipeline concluye con el **despliegue automático** de la aplicación en entornos de prueba o producción, según la configuración establecida.

El flujo general del pipeline puede representarse de la siguiente manera:

Developer → Pull Request → Build → Security Scan → Tests → Deploy

IV. Marco teórico

A. DevSecOps

DevSecOps es una evolución de la metodología DevOps que incorpora prácticas de seguridad a lo largo de todo el ciclo de vida del desarrollo de software [1]. Su objetivo es integrar controles de seguridad desde las primeras etapas del desarrollo, permitiendo detectar vulnerabilidades de forma temprana y reducir riesgos antes de que las aplicaciones lleguen a producción. Para ello, utiliza herramientas automatizadas de análisis, pruebas y monitoreo continuo.

B. Integración y Entrega Continua (CI/CD)

Los pipelines de Integración Continua y Entrega Continua (CI/CD) automatizan tareas como la compilación, ejecución de pruebas y despliegue de aplicaciones [2]. Estas prácticas permiten acelerar el proceso de desarrollo, reducir errores manuales y garantizar una mayor calidad del software mediante la validación continua de los cambios realizados por los desarrolladores.

C. GitHub Actions

GitHub Actions es una plataforma de automatización integrada en GitHub que permite implementar flujos de integración y entrega continua mediante la ejecución automática de tareas definidas en archivos de configuración denominados workflows. Estos workflows pueden activarse ante eventos específicos del repositorio, como la creación de pull requests, la realización de commits o la publicación de nuevas versiones.

Gracias a su integración nativa con Git, GitHub Actions facilita la automatización de procesos de compilación, ejecución de pruebas, análisis de seguridad y despliegue de aplicaciones, constituyendo una herramienta ampliamente utilizada en entornos DevOps y DevSecOps.

D. Inteligencia Artificial en el Desarrollo de Software

La Inteligencia Artificial ha comenzado a desempeñar un papel relevante en el desarrollo de software gracias a su capacidad para analizar grandes volúmenes de información y automatizar tareas complejas [3]. Entre sus aplicaciones más comunes se encuentran la revisión de código, la detección de errores, la generación de pruebas y la asistencia en la documentación técnica, contribuyendo a mejorar la productividad de los equipos de desarrollo.

E. Modelos de Lenguaje de Gran Escala (LLM)

Los Large Language Models (LLM) son modelos de inteligencia artificial entrenados sobre grandes cantidades de texto y código fuente [4]. Gracias a su capacidad para comprender contexto y generar respuestas coherentes, pueden utilizarse como asistentes para desarrolladores, proporcionando recomendaciones de código, identificando posibles vulnerabilidades y generando documentación de manera automática.

F. Ejecución local de modelos de IA

La ejecución local de modelos de lenguaje permite procesar información sin necesidad de enviarla a servicios externos. Herramientas como Ollama facilitan el despliegue y la utilización de modelos de código abierto mediante una interfaz de programación sencilla, favoreciendo escenarios donde la privacidad y la confidencialidad de la información resultan requisitos fundamentales.

G. Seguridad en Aplicaciones Web

Las aplicaciones web modernas procesan información sensible y se encuentran expuestas a diversas amenazas de seguridad. Organizaciones como OWASP publican guías y recomendaciones que ayudan a identificar vulnerabilidades comunes y establecer buenas prácticas de desarrollo seguro [5]. La incorporación de controles de seguridad dentro de los pipelines DevSecOps permite reducir la exposición a estos riesgos y fortalecer la protección de las aplicaciones.

V. Desarrollo

A. Análisis de caso

Tras analizar el caso de estudio presentado, se identificó que, si bien la organización cuenta con un pipeline DevSecOps que automatiza tareas de integración, testing y análisis de seguridad, aún persisten limitaciones relacionadas con la detección temprana de vulnerabilidades, la generación de pruebas automatizadas y la documentación del código nuevo.

En particular, muchas de estas tareas continúan dependiendo en gran medida de procesos manuales o de herramientas tradicionales que operan en etapas posteriores del pipeline. Esto provoca que ciertos problemas de seguridad o calidad del código sean detectados recién durante fases avanzadas del proceso de integración o incluso durante las revisiones manuales realizadas por los equipos de desarrollo o seguridad.

En este contexto, surge la oportunidad de mejorar el pipeline existente mediante la incorporación de técnicas basadas en inteligencia artificial que permitan analizar el código de manera más temprana dentro del flujo de integración continua.

Para abordar esta problemática, se propone la incorporación de

un módulo de inteligencia artificial dentro del pipeline DevSecOps, orientado a mejorar la detección temprana de vulnerabilidades, la generación automatizada de pruebas y la asistencia en la documentación del código.

La solución planteada se basa en la integración de LLM especializados en análisis de código, los cuales pueden ser ejecutados de forma local o mediante servicios controlados, garantizando así la privacidad de la información sensible.

La incorporación de este componente se ubica en etapas tempranas del pipeline, particularmente asociadas a la creación de pull requests, permitiendo analizar los cambios introducidos antes de que estos avancen en el flujo de integración. De esta manera, el pipeline propuesto adopta la siguiente estructura:

Developer → Pull Request→ AI Code Analysis → Build → Security Scan → Tests → Deploy

A partir de la incorporación del módulo de inteligencia artificial dentro del pipeline DevSecOps, resulta necesario definir las funcionalidades que este aportará al proceso de desarrollo y validación del software.

La solución propuesta se enfoca en tres capacidades principales orientadas a mejorar la calidad, seguridad y mantenibilidad del código desde etapas tempranas del ciclo de desarrollo.

Detección temprana de vulnerabilidades: analiza los cambios introducidos en el código para identificar posibles problemas de seguridad antes de avanzar hacia las siguientes etapas del pipeline.

Generación automática de pruebas: crea casos de prueba unitarios a partir de las modificaciones realizadas, con el objetivo de aumentar la cobertura de testing y reducir el esfuerzo manual asociado a esta tarea.

Generación de documentación contextual: produce descripciones y explicaciones sobre el código modificado, facilitando su comprensión, revisión y mantenimiento por parte del equipo de desarrollo.

Por lo que el pipeline finalmente quedaría de la siguiente forma

Developer → Pull Request→ AI Vuln analysis → AI Test Generation → AI Documentation → Build → Security Scan → Tests → Deploy

Si bien la solución propuesta contempla las tres funcionalidades mencionadas, debido al alcance y a los objetivos de la prueba de concepto desarrollada en este trabajo, la implementación se centra exclusivamente en el módulo de detección temprana de vulnerabilidades. Esta funcionalidad fue seleccionada por su impacto directo en la seguridad del pipeline DevSecOps y por permitir evaluar de forma práctica la integración de modelos de inteligencia artificial dentro del proceso de integración continua.

El análisis realizado por IA no reemplaza las herramientas tradicionales de análisis estático, sino que las complementa. Mientras la IA aporta detección temprana y análisis contextual, las herramientas de seguridad continúan actuando como mecanismo formal de validación dentro del pipeline.

Una vez definidas las funcionalidades, resulta necesario determinar qué modelos de inteligencia artificial serán utilizados para su implementación.

Para esta prueba de concepto se optó por emplear Qwen2.5-Coder 14B, un modelo especializado en tareas de programación que puede ejecutarse de forma local mediante Ollama.
La elección de este modelo se fundamenta en su capacidad para comprender la estructura y la lógica del código fuente, analizar cambios introducidos mediante diffs de Git e identificar potenciales vulnerabilidades de seguridad. Su ejecución local permite además preservar la confidencialidad del código analizado, evitando la necesidad de enviar información sensible a servicios externos.

Dentro del pipeline propuesto, el modelo recibe únicamente las modificaciones introducidas en el repositorio y genera una respuesta estructurada con las vulnerabilidades detectadas. Este enfoque permite incorporar capacidades de análisis inteligente en etapas tempranas del ciclo de desarrollo, proporcionando retroalimentación inmediata a los desarrolladores y fortaleciendo las prácticas DevSecOps sin sustituir los mecanismos tradicionales de validación de seguridad.

B. Arquitectura e integración

La arquitectura descrita a continuación corresponde a la implementación de la prueba de concepto desarrollada en este trabajo. En particular, se implementa el módulo de detección temprana de vulnerabilidades mediante inteligencia artificial, integrándolo dentro del pipeline de integración continua para analizar automáticamente los cambios introducidos en el código fuente.

Con el objetivo de preservar la confidencialidad del código fuente analizado, la solución propuesta utiliza un modelo de inteligencia artificial ejecutado localmente mediante Ollama. Este enfoque evita la necesidad de enviar información sensible a servicios externos y permite mantener el procesamiento dentro de la infraestructura controlada por la organización.

La integración con el pipeline de integración continua se realiza mediante GitHub Actions. Ante cada actualización del repositorio o solicitud de integración, el pipeline ejecuta automáticamente un script encargado de obtener las modificaciones realizadas en el código y enviarlas al modelo para su análisis.

Para minimizar el volumen de información procesada, la solución se basa en el análisis de diferencias (diffs) generadas por Git. De esta forma, el modelo recibe únicamente el código agregado o modificado en los cambios más recientes, evitando analizar la totalidad del proyecto. Este enfoque reduce

significativamente el consumo de recursos computacionales y los tiempos de ejecución, además de enfocar el análisis exclusivamente sobre los cambios introducidos por el desarrollador.

La comunicación con el modelo se realiza mediante la API HTTP expuesta por Ollama. El pipeline envía el diff generado junto con un conjunto de instrucciones predefinidas que describen la tarea de análisis a realizar. Posteriormente, la respuesta del modelo es procesada automáticamente para determinar si existen vulnerabilidades de seguridad que deban bloquear la integración del código.

Con el fin de mejorar la consistencia de los resultados, se emplean prompts estructurados que delimitan claramente el objetivo del análisis, las vulnerabilidades que deben ser consideradas y el formato esperado de la respuesta. En particular, el modelo es instruido para identificar riesgos comunes de seguridad, tales como inyecciones SQL, Cross-Site Scripting (XSS), exposición de información sensible, problemas de autenticación y autorización, configuraciones inseguras, Command Injection, Path Traversal y SSRF, entre otros.

Asimismo, se establece como requisito que la salida sea generada en formato JSON estructurado. Esto permite que el pipeline interprete automáticamente los resultados y determine si el proceso debe continuar o bloquearse en función de las vulnerabilidades detectadas.

La estandarización de los prompts y de los formatos de salida contribuye a reducir la variabilidad inherente a los modelos de lenguaje y favorece la reproducibilidad de los análisis. Este aspecto resulta especialmente relevante en entornos DevSecOps, donde las decisiones automatizadas deben ser consistentes, auditables y fácilmente integrables con los mecanismos tradicionales de aseguramiento de la calidad y la seguridad.

VI. Conclusión

Las organizaciones que adoptan prácticas DevSecOps buscan mejorar la calidad y seguridad del software mediante la automatización de procesos. Sin embargo, tareas como la revisión de código, la generación de pruebas y la detección temprana de errores continúan requiriendo una participación significativa de los desarrolladores, lo que puede generar demoras y permitir que ciertos problemas lleguen a etapas avanzadas del ciclo de desarrollo.

Para abordar esta situación, en este trabajo se analizó la incorporación de herramientas de Inteligencia Artificial dentro de un pipeline DevSecOps. A partir de un caso de estudio basado en una organización dedicada al desarrollo de aplicaciones web para el sector financiero, se identificaron oportunidades para integrar capacidades de análisis de código, complementando los mecanismos tradicionales de desarrollo seguro.

El análisis realizado permitió concluir que la Inteligencia Artificial puede aportar valor al proceso DevSecOps al facilitar la detección temprana de errores y vulnerabilidades, reducir tareas manuales repetitivas y proporcionar retroalimentación más rápida a los equipos de desarrollo. Si bien estas herramientas no reemplazan la supervisión humana, su utilización resulta especialmente útil en equipos con desarrolladores menos experimentados, contribuyendo a mejorar la calidad del software y fortalecer la seguridad de las aplicaciones desde las primeras etapas del desarrollo.

VII. Líneas futuras de investigación

Evaluar y comparar distintos modelos de IA en tareas de análisis de código, generación de pruebas y documentación automática.

Analizar la adaptación de los modelos a proyectos específicos para mejorar la precisión de las recomendaciones generadas.

Incorporar capacidades de análisis de infraestructura como código (IaC) dentro del pipeline DevSecOps.

VIII. Referencias

[1] G. Kim, J. Humble, P. Debois y J. Willis, The DevOps Handbook. Portland, OR, USA: IT Revolution Press, 2016.

[2] J. Humble y D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.

[3] T. Davenport y R. Miller, "Working with AI: Generative AI and the Future of Software Development," IEEE Software, vol. 41, no. 2, pp. 10-17, 2024.

[4] OpenAI, "GPT-4 Technical Report," 2023. Disponible en: https://arxiv.org/abs/2303.08774

[5] OWASP Foundation, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021. Disponible en: https://owasp.org/www-project-top-ten/

IX. Curriculum vitae

Soy Francisco Noceda, estudiante y desarrollador backend con interés en devops e inteligencia artificial. Experiencia en desarrollo de APIs, bases de datos relacionales, testing de servicios y uso de herramientas modernas de automatización y despliegue. Conocimientos en prácticas de entrega continua, seguridad en aplicaciones web y exploración de herramientas de IA generativa aplicadas al desarrollo.